

GOOGLE CAREER / INTERVIEW BLUEPRINT

DAY 2 — INTERVIEW QUESTIONS & ANSWERS

Textbook-style reference notes • Friday, 11 September 2026

Theme: From Python business logic → real API service

Scope: Two Pointers DSA • FastAPI • Uvicorn • Pydantic • API contracts • API → Service → Repository • Error handling • pytest/API testing • virtual environments • Git discipline • project reasoning

Important source note: The uploaded Day 2 plan defines the tasks, architecture, checklist, success criteria, and deliberately postponed technologies. This book expands those Day 2 topics into interview-oriented questions and answers. Where the plan gives only a task rather than an implementation detail, the answers explain the underlying engineering concept rather than pretending that a specific implementation was present in the plan.

Table of Contents

1. 1. How to Use These Notes
2. 2. Day 2 Executive Summary
3. 3. Backend Project Setup & Python Environment
4. 4. FastAPI Fundamentals
5. 5. HTTP, REST and API Design
6. 6. Pydantic and API Contracts
7. 7. API Schema vs Business Object vs Database Model
8. 8. API → Service → Repository Architecture
9. 9. FastAPI Error Handling
10. 10. API Testing and pytest
11. 11. DSA Pattern Recognition — Two Pointers
12. 12. Valid Palindrome
13. 13. Two Sum II
14. 14. Best Time to Buy and Sell Stock
15. 15. Git and GitHub Engineering Discipline
16. 16. Project Structure and Engineering Trade-offs
17. 17. End-to-End Day 2 Scenarios
18. 18. What You Deliberately Did Not Build
19. 19. High-Priority Mock Interview
20. 20. Rapid-Fire Revision
21. 21. Day 2 Final Revision Checklist

1. How to Use These Notes

These notes are designed as an interview reference after completing Day 2. The objective is not to memorize definitions mechanically. You should be able to explain what you built, why the architecture is shaped that way, what happens on a request, how errors move across the boundary, how the API is tested, and why the DSA optimizations are correct.

The uploaded Day 2 plan states that the central progression is: yesterday's pure-Python business logic is reused and exposed through FastAPI. It also defines the target request flow as HTTP Client → FastAPI Router → DocumentService → DocumentRepository → In-Memory Store. ■filecite■turn0file0■

Interview rule: For every technical decision, practice answering three questions: (1) What is it? (2) Why did you use it? (3) What would change if the system grew?

Important limitation: "All possible interview questions" cannot literally be finite because interviewers can generate unlimited variations. This document therefore aims for a comprehensive Day 2 question bank covering every task, checklist item, architecture decision, DSA problem, and engineering takeaway in the Day 2 plan, plus natural follow-up questions derived from those topics.

2. Day 2 Executive Summary

Day 2 was deliberately a carry-forward day. The plan records that DSA and Python were completed on Day 1 while the Project 1 skeleton and FastAPI setup were still pending. Therefore, Day 2 prioritizes the backend foundation rather than adding another standalone Python topic. ■filecite■turn0file0■



1. What was the primary goal of Day 2?

Answer: The goal was to finish the Day 1 project carry-forward and expose the Python DocumentService through FastAPI, turning the existing Python business logic into a working local backend service.

2. What was the key engineering progression on Day 2?

Answer: The progression was from Python business logic to a real HTTP API: the API layer accepts and validates HTTP input, the service layer owns business behavior, and the repository layer owns storage operations.

3. Why was no new standalone Python topic added?

Answer: Because the Day 2 plan intentionally treats the previous Python work as the business-logic foundation of Project 1. The priority was integration, not rewriting or expanding the Python exercise. ■filecite■turn0file0■

4. What did “done” mean at the end of Day 2?

Answer: The plan defines success as working GET / and GET /health endpoints, POST /documents reaching DocumentService and DocumentRepository, creating a document in in-memory storage, rejecting invalid input, mapping missing documents to 404, passing pytest, and pushing both repositories to GitHub. ■filecite■turn0file0■

3. Backend Project Setup & Python Environment

The Day 2 project is named **ai-engineering-intelligence-platform**. The plan asks for a clean foundation with src, api, services, models, repositories, core, tests and docs, plus requirements.txt, .gitignore and README.md. It explicitly says not to over-engineer the structure. ■filecite■turn0file0■

```
ai-engineering-intelligence-platform/
├── src/
│   ├── main.py
│   ├── api/
│   ├── services/
│   ├── models/
│   ├── repositories/
│   └── core/
├── tests/
├── docs/
├── requirements.txt
├── .gitignore
└── README.md
```

5. What is a Python virtual environment?

Answer: A virtual environment is an isolated Python environment for a project. It allows project-specific packages and versions to be installed without mixing them with packages used by other projects or the system Python installation.

6. Why create a virtual environment for this project?

Answer: FastAPI, Uvicorn, pytest and future dependencies belong to the project environment. Isolation makes the project reproducible and reduces dependency conflicts.

7. What does `python -m venv .venv` do?

Answer: It creates a virtual environment in a directory named `.venv` using Python's built-in venv mechanism.

8. Why should `.venv` normally be in `.gitignore`?

Answer: The environment contains installed packages and machine-specific files. It should be recreated from dependency declarations rather than committed into source control.

9. What is pip?

Answer: pip is Python's package installer. It is used to install and manage Python packages such as FastAPI, Uvicorn and pytest.

10. What is requirements.txt?

Answer: It is a dependency declaration file listing packages required by the project, typically with versions when reproducibility matters.

11. What does `pip freeze > requirements.txt` do?

Answer: It writes the packages installed in the active environment, with their versions, into requirements.txt. This provides a snapshot of the current environment.

12. What is the difference between `python --version` and `pip --version`?

Answer: `python --version` verifies the Python interpreter being used. `pip --version` verifies the pip installation and, importantly, can help confirm which Python environment pip belongs to.

13. Why verify that pytest is available?

Answer: Because automated tests are a required part of Day 2. Verifying pytest early catches environment or installation problems before the test phase.

14. What belongs in .gitignore for a Python project?

Answer: Typical entries include the virtual environment, Python bytecode/cache directories, test caches, IDE metadata, local secrets and other generated files. The exact file should reflect the project rather than blindly ignoring everything.

15. Why is environment reproducibility important in interviews?

Answer: It demonstrates that the project is an engineering artifact rather than code that only works on one machine. A reviewer should be able to understand and recreate the dependencies.

4. FastAPI Fundamentals

Day 2 requires a minimal FastAPI application with GET /, GET /health and POST /documents. The application is run with Uvicorn and should be discoverable through FastAPI's interactive documentation at /docs. ■filecite■turn0file0■

```
uvicorn src.main:app --reload

GET /
{
  "application": "AI Engineering Intelligence Platform",
  "status": "running",
  "version": "0.1.0"
}

GET /health
{
  "status": "healthy"
}
```

16. What is FastAPI?

Answer: FastAPI is a Python web framework for building APIs. In this Day 2 project it provides the HTTP boundary, routing, request parsing, validation integration and API documentation facilities.

17. Why use FastAPI for this project?

Answer: It provides a concise way to expose Python business logic as HTTP endpoints while working naturally with Python type hints and Pydantic models. That makes it appropriate for demonstrating API contracts and layered backend architecture.

18. What is Uvicorn?

Answer: Uvicorn is an ASGI server used to run Python ASGI applications such as FastAPI applications.

19. What does `uvicorn src.main:app --reload` mean?

Answer: It tells Uvicorn to import the `app` object from `src.main` and serve it. `--reload` enables automatic server reload during local development when source files change.

20. What is ASGI?

Answer: ASGI is the Asynchronous Server Gateway Interface, a standard interface between Python web applications/frameworks and compatible servers. FastAPI applications run through an ASGI server such as Uvicorn.

21. What is an endpoint?

Answer: An endpoint is a defined API operation exposed at a particular HTTP method and path, such as GET /health or POST /documents.

22. What is a route?

Answer: A route maps an HTTP method and URL path to application code that handles the request.

23. Why have a GET / endpoint?

Answer: It provides a simple application-level endpoint that can confirm the service is running and return basic application metadata.

24. Why have a /health endpoint?

Answer: A health endpoint provides a simple signal about application health. It can later be used by deployment or infrastructure systems to check whether the service is responding.

25. What is `/docs` in FastAPI?

Answer: It is the interactive API documentation interface generated from the application's route and schema definitions. It allows a developer to inspect and try endpoints.

26. What is Swagger UI?

Answer: Swagger UI is an interactive interface for exploring and trying APIs described by an OpenAPI specification. FastAPI provides a Swagger UI documentation endpoint by default.

27. How does FastAPI know which function handles a request?

Answer: Route declarations associate an HTTP method and path with a Python function. When a matching request arrives, FastAPI dispatches it to that handler.

28. What happens conceptually when an HTTP request reaches FastAPI?

Answer: The server receives the request, FastAPI matches the method/path to a route, request data is parsed and validated as required, the route handler runs, the application produces a result or error, and FastAPI constructs an HTTP response.

29. What is the difference between development reload and production serving?

Answer: `--reload` is a developer convenience that restarts/reloads the application when source code changes. Production deployments generally use an appropriate process/server configuration rather than relying on development reload.

5. HTTP, REST and API Design

30. What is HTTP?

Answer: HTTP is the protocol used for communication between clients and servers on the web. APIs commonly use HTTP methods, status codes, headers and representations such as JSON.

31. What is the difference between GET and POST?

Answer: GET is generally used to retrieve a representation or resource without requesting a state-changing operation. POST is commonly used to submit data for processing or create a subordinate resource.

32. Why is POST `/documents` appropriate for document creation?

Answer: The endpoint represents a creation operation where the client submits document attributes to the server. POST is the conventional HTTP method for such creation semantics.

33. What is a request body?

Answer: The request body contains data sent by the client as part of an HTTP request. In Day 2, the document creation endpoint receives `name`, `document_type` and `version` in JSON.

34. What is a path parameter?

Answer: A path parameter is a value embedded in the URL path, commonly used to identify a specific resource, such as `/documents/{document_id}`.

35. What is a query parameter?

Answer: A query parameter is a key-value parameter supplied in the URL query string, commonly used for filtering, searching, sorting or optional controls.

36. When would you use a request body instead of query parameters?

Answer: For structured data that represents a submitted resource or command, especially when multiple related fields are being supplied, a request body is generally more appropriate.

37. What does HTTP 200 mean?

Answer: It means the request succeeded.

38. What does HTTP 201 mean?

Answer: It means a resource was successfully created. It is commonly appropriate for successful POST resource creation, although the Day 2 plan focuses on proving the API behavior rather than requiring a specific creation status code.

39. What does HTTP 400 mean?

Answer: It generally indicates a bad request where the server cannot or should not process the request because the client sent invalid input or violated a request-level expectation.

40. What does HTTP 404 mean?

Answer: It means the requested resource was not found. Day 2 explicitly maps `DocumentNotFoundError` to HTTP 404. ■filecite■turn0file0■

41. What does HTTP 4xx represent?

Answer: 4xx responses generally indicate a client-side/request-side problem, such as invalid input or a resource that cannot be found.

42. What does HTTP 5xx represent?

Answer: 5xx responses generally indicate that the server failed to fulfill an otherwise valid request because of an application or infrastructure problem.

43. What is JSON?

Answer: JSON is a text-based data interchange format commonly used by HTTP APIs. It represents objects, arrays, strings, numbers, booleans and null in a standardized syntax.

6. Pydantic and API Contracts

Day 2 introduces explicit request and response models instead of treating raw dictionaries as the long-term API contract. The conceptual request model is `DocumentCreate` with `name`, `document_type` and `version`; the response model is `DocumentResponse` with `id`, `name`, `document_type`, `version` and `status`. ■filecite■turn0file0■

```
DocumentCreate
- name
- document_type
- version
```

```
DocumentResponse
- id
- name
- document_type
- version
- status
```

44. What is Pydantic?

Answer: Pydantic is a Python library for defining data models using Python type annotations and validating/parsing data against those models. FastAPI integrates with it for request and response schemas.

45. Why use Pydantic with FastAPI?

Answer: It makes the HTTP contract explicit and gives the framework a structured way to validate and represent incoming and outgoing data.

46. What is a Pydantic model?

Answer: It is a class that describes the expected structure and types of data. Input data can be parsed and validated against that structure.

47. Why are explicit schemas better than raw dictionaries?

Answer: A schema documents the contract, makes expected fields and types explicit, enables validation, improves editor/type-checking support, and reduces ambiguity about what the API accepts and returns.

48. What is validation?

Answer: Validation is the process of checking whether input satisfies defined structural, type and business-independent constraints before the application proceeds with it.

49. What is serialization?

Answer: Serialization is converting an in-memory representation into a representation suitable for transport or storage, such as JSON-compatible data.

50. What is deserialization?

Answer: Deserialization is converting an external representation such as JSON into an in-memory representation that application code can work with.

51. What happens if a required Pydantic field is missing?

Answer: The data does not satisfy the declared schema, so FastAPI/Pydantic validation rejects the request and returns an appropriate validation error response.

52. What happens if a field has an invalid type?

Answer: Validation/parsing attempts to interpret the value according to the declared type. If it cannot satisfy the schema, the request is rejected with a validation error.

53. What is DocumentCreate?

Answer: It is the API request schema for document creation. It defines the data the client is expected to send at the HTTP boundary: name, document_type and version.

54. What is DocumentResponse?

Answer: It is the API response schema for a document returned by the service. It defines the response contract including id, document fields and status.

55. Why separate request and response models?

Answer: The client's input and the server's output have different responsibilities. The server may generate fields such as an ID or status that the client should not provide. Separate models also make API evolution safer.

56. Why not use raw dictionaries everywhere?

Answer: Dictionaries do not express a stable contract by themselves. They allow missing keys, unexpected keys and inconsistent value shapes unless additional validation is manually written.

57. What are Python type hints doing here?

Answer: They describe expected data types. FastAPI and Pydantic can use these declarations to generate schemas, validate input and make the code easier to understand.

58. Does Pydantic validation replace business validation?

Answer: No. Pydantic is appropriate for API data structure/type/schema validation. Domain-specific business rules belong in the service/business layer.

59. Where should a rule like “document version must follow a domain-specific policy” live?

Answer: If it is a business rule, it should be owned by the service/domain logic rather than being hidden inside the HTTP route. Some simple field constraints can be represented in the schema, but domain behavior should remain in the business layer.

7. API Schema vs Business Object vs Database Model

One of the most important Day 2 concepts is: **API schema** ≠ **Database model** ≠ **Business object**. The plan explicitly defines API schemas as the HTTP boundary, business objects as domain/business logic representations, and database models as persistence representations. ■filecite■turn0file0■

60. What is an API schema?

Answer: It defines what enters or leaves the HTTP boundary. It is designed around the external API contract.

61. What is a business object?

Answer: It represents domain concepts and behavior used by the application's business logic.

62. What is a database model?

Answer: It represents how information is persisted in a storage system and is shaped by persistence concerns.

63. Why are these three concepts different?

Answer: They change for different reasons. An API changes because clients and contracts evolve, business objects change because business rules evolve, and database models change because persistence requirements evolve.

64. Why can using one class for all three become problematic?

Answer: A single class can accumulate HTTP concerns, domain behavior and database-specific details. That creates tight coupling, makes changes risky, and exposes internal representation through the API.

65. Give an example of a field that might exist in a database model but not in an API response.

Answer: An internal persistence identifier, audit metadata or storage-specific field may be useful internally but unnecessary or inappropriate for external clients.

66. Give an example of a field generated by the server.

Answer: A document ID is a natural example: the client supplies document attributes while the application generates the identifier.

67. What happens if the database changes but the API contract should remain stable?

Answer: The repository/persistence layer should absorb most of the change. The API schema can remain stable if the business meaning and external contract have not changed.

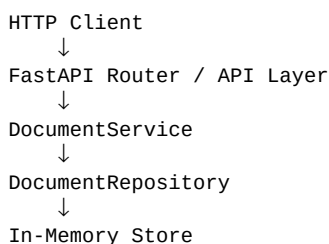
68. What happens if the API response changes but the database does not?

Answer: The API response model can change without forcing a persistence schema change, provided the required information can be produced from the existing business/persistence data.

69. Why does separation make systems easier to evolve?

Answer: It limits the number of components affected by a change. Each layer has a clearer responsibility and a smaller contract with the layers around it.

8. API → Service → Repository Architecture



The Day 2 plan describes this as the most important architectural step. The API translates HTTP input into a business operation; the service continues to own business rules; the repository continues to own storage operations. The plan also explicitly says not to rewrite Document, DocumentRepository, DocumentService or the custom exceptions simply to make them fit FastAPI. ■filecite■turn0file0■

70. Explain the architecture of the Day 2 project.

Answer: A client sends an HTTP request to the FastAPI API layer. The API validates/translate the request and calls DocumentService. The service applies business behavior and calls DocumentRepository for persistence. The repository interacts with the in-memory store. The result is then converted into the API response contract.

71. What belongs in the API layer?

Answer: HTTP-specific concerns: routing, request/response schemas, HTTP status codes, translating application errors into HTTP behavior, and other transport-level concerns.

72. What belongs in the service layer?

Answer: Business behavior and business rules. The service should operate on domain concepts rather than being tightly coupled to HTTP.

73. What belongs in the repository layer?

Answer: Persistence/storage operations: creating, retrieving and otherwise manipulating stored data without owning HTTP semantics.

74. Why should business rules not live in FastAPI routes?

Answer: Routes are transport handlers. Putting business rules there couples the business logic to HTTP, makes reuse harder, and makes testing more difficult.

75. Why should the repository not return HTTP responses?

Answer: HTTP is an API-layer concern. If the repository knows about status codes, it becomes coupled to one transport mechanism and cannot be cleanly reused by other callers.

76. Why should the service not return HTTP responses?

Answer: The service represents business behavior and should not need to know whether its caller is HTTP, a command-line program, a background worker or another internal component.

77. Why use a repository if storage is only in memory?

Answer: Even with simple storage, the repository establishes a persistence boundary. Later, storage can be changed without forcing the service and API layers to own storage details.

78. Why use a service if the endpoint currently performs only a simple operation?

Answer: The service establishes a stable location for business rules. As the application grows, rules remain outside the HTTP layer rather than accumulating in routes.

79. What is separation of concerns?

Answer: It is the practice of separating responsibilities so each component focuses on a coherent concern. In Day 2, HTTP, business logic and persistence are separated.

80. How does this architecture improve testing?

Answer: Each layer can be tested according to its responsibility. API tests can verify the HTTP contract, service tests can verify business rules, and repository tests can verify storage behavior.

81. How would PostgreSQL fit into this architecture later?

Answer: The repository implementation would become responsible for PostgreSQL persistence. The API contract and service responsibilities should remain conceptually separate from database-specific details.

82. What is dependency injection in this context?

Answer: Dependency injection means supplying a component's dependencies from outside rather than having the component construct them internally. For example, a service can receive a repository implementation.

83. Why is dependency injection useful?

Answer: It makes components easier to test, replace and configure because their dependencies are explicit rather than hidden.

84. What is a good interview explanation of the architecture in one sentence?

Answer: "FastAPI owns transport, the service owns business behavior, and the repository owns persistence, so each layer can evolve and be tested independently."

9. FastAPI Error Handling

Day 2 requires Python exceptions to become meaningful HTTP responses. The mental model is: Python exception → API error handling → HTTP status code + response body. The explicit example is `DocumentNotFoundError` → HTTP 404. ■filecite■turn0file0■

85. What is a Python exception?

Answer: An exception is an object representing an abnormal condition during program execution. It can be raised by application code and handled by an appropriate caller.

86. What is an HTTP error response?

Answer: It is an HTTP response with a status code and body that communicates that the request could not be completed as requested.

87. What is the difference between a Python exception and an HTTP error?

Answer: A Python exception is an internal application mechanism. An HTTP error is an external protocol-level representation. The API layer translates between them when appropriate.

88. How does `DocumentNotFoundError` become 404?

Answer: The service/repository can raise the domain/application exception when the document is absent. The API layer catches or maps that exception to an HTTP 404 response.

89. Why is 404 appropriate for a missing document?

Answer: The requested resource does not exist from the API client's perspective, which is exactly the meaning of 404 Not Found.

90. What should happen for invalid client input?

Answer: The request should be rejected with an appropriate client/validation response. Day 2 requires invalid input to be rejected rather than silently accepted.

91. What should happen for an unexpected application failure?

Answer: It should not be converted into a fake success response. It should be surfaced through appropriate error handling, generally resulting in a server-error response if it cannot be handled as a known client/domain condition.

92. Why should HTTP-specific behavior stay in the API layer?

Answer: Because HTTP status codes, HTTP response bodies and transport semantics belong to the API boundary. Keeping them there prevents domain and persistence code from becoming transport-dependent.

93. What is `HTTPException` in FastAPI?

Answer: It is a FastAPI mechanism for producing an HTTP error response with a status code and detail information from application handling code.

94. What is the difference between 404 and validation errors?

Answer: 404 means the requested resource was not found. A validation error means the request data did not satisfy the declared input contract. They represent different failure causes.

95. Why not catch every exception and return 200?

Answer: That hides failures, tells clients the operation succeeded when it did not, makes debugging harder and violates the meaning of HTTP status codes.

96. Where should a domain exception be translated into HTTP?

Answer: At the API boundary, because that is where domain/application outcomes are converted into transport semantics.

10. API Testing and pytest

Day 2 requires tests proving that the API contract and architecture work. The required behaviors are GET / → 200, GET /health → 200 with healthy status, successful POST /documents, invalid request → validation error, missing document → 404, and a passing pytest run. ■filecite■turn0file0■

97. What is pytest?

Answer: pytest is a Python testing framework used to discover and execute tests and report assertions and failures.

98. Why use pytest?

Answer: It provides a simple syntax, automatic test discovery, fixtures and a strong ecosystem for Python testing.

99. How does pytest discover tests?

Answer: By conventional naming patterns for test files and test functions/methods. Standard pytest conventions make test discovery automatic.

100. What is an assertion?

Answer: An assertion expresses an expected condition. If the condition is false, the test fails.

101. What is a fixture?

Answer: A fixture provides reusable setup or data for tests. Fixtures are useful for preparing application objects, test clients or isolated state.

102. Why should tests be independent?

Answer: Independent tests are predictable and easier to debug. One test should not rely on another test having run first or leaving behind specific mutable state.

103. How do you test GET /?

Answer: Send a GET request to the endpoint and assert that the response has the expected success status and expected response fields.

104. How do you test GET /health?

Answer: Send a GET request, assert a successful status, and verify that the returned status field indicates healthy.

105. How do you test POST /documents?

Answer: Send a valid JSON request body matching DocumentCreate, assert success, and verify that the returned response contains the expected document data and server-generated fields.

106. How do you test invalid input?

Answer: Send a request that violates the declared API schema, such as omitting a required field or supplying invalid data, and assert that the API rejects it with a validation/client error.

107. How do you test a missing document?

Answer: Invoke the API operation that attempts to retrieve a document that does not exist and assert that the response is 404.

108. Why test status codes and response bodies?

Answer: A status code verifies the protocol-level contract while the body verifies the data contract. Both are part of observable API behavior.

109. What is an API test?

Answer: An API test exercises the HTTP boundary and verifies externally observable request/response behavior.

110. What is a unit test?

Answer: A unit test focuses on a small unit of behavior in relative isolation, such as a service method.

111. What is an integration test?

Answer: An integration test verifies that multiple components work together. A test that exercises FastAPI through its API boundary and uses the service/repository path can be considered integration-oriented.

112. Why not manually test only through Swagger?

Answer: Manual testing is useful for exploration, but automated tests are repeatable and executable. They prevent regressions and provide evidence that the API contract continues to work.

113. What does it mean that API behavior is “executable and repeatable”?

Answer: It means the expected behavior is encoded in tests that can be run repeatedly by developers or CI instead of relying on memory or manual clicking.

114. What is test isolation?

Answer: Test isolation means each test has controlled dependencies and state so its result is not accidentally affected by other tests.

115. What is mocking?

Answer: Mocking replaces a real dependency with a controlled test double so a test can focus on a particular unit of behavior.

116. When would you mock a repository?

Answer: When testing service behavior independently from persistence, especially when you want deterministic repository responses or want to verify how the service reacts to repository behavior.

117. What is the risk of excessive mocking?

Answer: Tests can become coupled to implementation details and may pass even when real components do not integrate correctly. The appropriate level of testing should include both isolated tests and meaningful integration/API tests.

11. DSA Pattern Recognition — Two Pointers

Day 2's DSA goal is pattern recognition rather than solving many problems. The three planned problems are Valid Palindrome, Two Sum II and Best Time to Buy and Sell Stock. For each, the plan asks for the problem statement, pattern, brute-force idea, optimized approach, time complexity, space complexity, why the optimized approach works, and a memorable mistake/observation. ■filecite■turn0file0■

118. What is the Two Pointer technique?

Answer: It is an algorithmic pattern where two indices/references are maintained to reduce unnecessary repeated work, often by exploiting ordering, boundaries or a relationship between elements.

119. Does Two Pointers always require a sorted array?

Answer: No. Sorting is especially important for problems such as Two Sum II, but other Two Pointer problems, such as palindrome checking, use the structure of the input rather than sorted order.

120. What are opposite-direction Two Pointers?

Answer: Two pointers begin at opposite ends of a sequence and move inward based on comparisons or conditions.

121. What are same-direction or fast/slow pointers?

Answer: Two pointers move in the same general direction at different speeds or with different roles. This is another family of pointer techniques and should not be confused with the opposite-end strategy.

122. Why can Two Pointers reduce $O(n^2)$ work?

Answer: Instead of examining every pair independently, pointer movement can eliminate entire ranges of candidates based on information already learned. Each pointer often moves only a limited number of times.

123. How do you recognize a Two Pointer problem?

Answer: Look for a sequence where two positions have a useful relationship: compare both ends, find a pair under sorted ordering, merge or partition data, or maintain two boundaries while scanning.

124. What is the key proof idea behind pointer movement?

Answer: You need to show that the candidates skipped by moving a pointer cannot contain a better/valid answer under the problem's assumptions. This is what makes the optimization correct rather than merely faster.

125. What is an invariant?

Answer: An invariant is a property that remains true throughout the algorithm. It helps explain why the algorithm's current state still represents all relevant possibilities.

126. What are common Two Pointer mistakes?

Answer: Moving the wrong pointer, using incorrect loop boundaries, forgetting to stop when pointers cross, mishandling duplicates or normalization, and claiming $O(n)$ without proving that pointer movement is bounded.

12. Valid Palindrome

The core idea is to compare characters from both ends and move inward. Depending on the exact problem definition, spaces, punctuation and letter case may need to be ignored. The important interview skill is explaining the normalization and the pointer invariant.

```
left = 0
right = len(s) - 1

while left < right:
    # skip characters that should be ignored
    # normalize case when required
    if normalized(s[left]) != normalized(s[right]):
        return False
    left += 1
    right -= 1

return True
```

127. What is a palindrome?

Answer: A palindrome is a sequence that reads the same forward and backward under the comparison rules of the problem.

128. What is the brute-force approach?

Answer: Normalize the string as required, reverse it, and compare it with the normalized original. This is straightforward but may allocate an additional string.

129. What is the Two Pointer approach?

Answer: Maintain a left pointer at the beginning and a right pointer at the end. Compare the relevant characters, then move inward until the pointers meet or cross.

130. Why do the pointers start at opposite ends?

Answer: A palindrome is defined by symmetry around the center. Comparing mirrored positions naturally maps to the beginning and end moving inward.

131. What happens when the two relevant characters are equal?

Answer: The algorithm can safely move both pointers inward because that mirrored pair satisfies the palindrome condition.

132. What happens when they differ?

Answer: The string cannot satisfy the palindrome condition, so the algorithm can return False immediately.

133. How do you handle spaces and punctuation?

Answer: Apply the normalization rule required by the problem, typically skipping non-relevant characters while locating the next comparable character.

134. How do you handle uppercase and lowercase?

Answer: Normalize the comparison, commonly by comparing lowercase forms or using a case-insensitive comparison rule.

135. What is the time complexity?

Answer: $O(n)$, because the pointers move through the string at most a linear number of times.

136. What is the space complexity?

Answer: $O(1)$ extra space if normalization is performed in place conceptually through pointer skipping rather than building a cleaned copy; $O(n)$ if a separate normalized string is created.

137. What edge cases should you test?

Answer: Empty input, one character, strings with only ignored characters, mixed case, punctuation/spaces, an early mismatch, and a valid palindrome.

138. What is the key invariant?

Answer: All relevant character pairs outside the current [left, right] region have already been verified to match.

13. Two Sum II

Two Sum II is the canonical Day 2 example where sorted input enables the opposite-direction Two Pointer technique. Start with one pointer at the smallest value and one at the largest value.

```
left = 0
right = len(numbers) - 1

while left < right:
    total = numbers[left] + numbers[right]

    if total == target:
        return [left + 1, right + 1]
    elif total < target:
        left += 1
    else:
        right -= 1
```

139. What is Two Sum II?

Answer: It asks for two numbers in a sorted array whose sum equals a target, typically returning their positions according to the problem's indexing convention.

140. Why is sorting important?

Answer: Sorted order gives directional information. If the current sum is too small, increasing the left value is the only direction that can increase the sum; if the sum is too large, decreasing the right value is the direction that can reduce it.

141. What is the brute-force approach?

Answer: Check every pair of elements until a pair reaches the target. That takes $O(n^2)$ time.

142. What is the optimized approach?

Answer: Use left and right pointers. Compare their sum to the target and move one pointer based on whether the sum is too small or too large.

143. Why move left when the sum is too small?

Answer: Because the array is sorted. Keeping the same left value and moving right leftward would only make the sum smaller. To increase the sum, the left pointer must move to a larger value.

144. Why move right when the sum is too large?

Answer: Because the array is sorted. Keeping the same right value and moving left upward would make the sum larger. To reduce the sum, the right pointer must move to a smaller value.

145. Why is it safe to discard candidates when moving a pointer?

Answer: The sorted order proves that the discarded candidates cannot produce the target with the current opposite pointer or with positions that have already been ruled out.

146. What is the time complexity?

Answer: $O(n)$, because each pointer moves only forward/inward through the array and together they perform a bounded linear number of movements.

147. What is the space complexity?

Answer: $O(1)$ extra space for the pointer variables, assuming the input is already sorted and no auxiliary structure is required.

148. Why not use a hash map?

Answer: A hash map is a common solution for unsorted Two Sum, but Two Sum II gives sorted input, so the ordering can be exploited to achieve linear time with constant extra space.

149. What happens if the input is unsorted?

Answer: The Two Sum II pointer proof no longer applies directly. A hash map can solve unsorted Two Sum in expected $O(n)$ time, or the array can be sorted with additional consideration for preserving original indices.

150. What is the cost of sorting an unsorted array?

Answer: Comparison sorting generally costs $O(n \log n)$, after which a linear Two Pointer scan can be performed. If original indices are required, the implementation must preserve them while sorting or use another strategy.

151. What happens with duplicates?

Answer: Duplicates are allowed depending on the problem constraints. The pointers still work because they represent positions, not unique values.

152. What if no pair exists?

Answer: The algorithm reaches the termination condition without finding a valid pair. The exact return behavior depends on the problem specification.

14. Best Time to Buy and Sell Stock

The Day 2 plan groups this problem with the Two Pointer set but specifically describes the optimized idea as tracking the minimum price seen so far and the best profit. This is an important interview lesson: pattern families can be related even when the implementation is not literally two moving indices. ■filecite■turn0file0■

```
min_price = infinity
best_profit = 0

for price in prices:
    min_price = min(min_price, price)
    best_profit = max(best_profit, price - min_price)

return best_profit
```

153. What is the brute-force approach?

Answer: Try every possible buy day before every possible sell day, compute the profit, and keep the maximum. This takes $O(n^2)$ time.

154. What is the optimized approach?

Answer: Scan once while remembering the minimum price seen so far and the best profit achieved so far.

155. Why is tracking only the minimum price sufficient?

Answer: For a given selling day, the best possible earlier buying price is the minimum price seen before that day. There is no need to remember every earlier price individually.

156. Why must buying occur before selling?

Answer: The problem requires the transaction to occur in chronological order. A future price cannot be used as the buying price for an earlier selling day.

157. What happens when prices continuously decrease?

Answer: No profitable transaction exists, so the best profit remains zero under the usual problem definition.

158. What is the time complexity?

Answer: $O(n)$, because the prices are scanned once.

159. What is the space complexity?

Answer: $O(1)$ extra space because only a small fixed number of variables are maintained.

160. What invariant does the algorithm maintain?

Answer: After processing each position, `min_price` is the minimum price seen up to that point and `best_profit` is the maximum valid profit obtainable using processed prices.

161. Is this literally an opposite-direction Two Pointer algorithm?

Answer: No. The Day 2 plan groups it with the Two Pointer practice set, but its described optimized solution is a one-pass state-tracking approach. In an interview, be precise about the actual mechanism rather than forcing every problem into one implementation pattern.

162. Why does the one-pass algorithm beat $O(n^2)$?

Answer: It summarizes all earlier buying opportunities with the single most useful value: the minimum price seen so far. That eliminates the need to compare the current selling price against every earlier day.

163. What changes if multiple transactions are allowed?

Answer: That becomes a different problem with different state transitions and rules. The single-transaction invariant is no longer sufficient by itself.

164. What if there is a transaction fee or cooldown?

Answer: Those constraints change the state required by the algorithm. They lead to variants commonly solved with dynamic-programming-style state tracking.

15. Git and GitHub Engineering Discipline

The Day 2 plan asks for meaningful commits instead of one giant end-of-day commit. Its suggested sequence includes chore: initialize backend project structure; feat: add FastAPI application and health endpoints; feat: add document creation API; refactor: connect document service to API layer; test: add document API tests; and feat: add two pointer DSA solutions. It also asks you to push both repositories and verify README, source tree, tests and commit history.

filecite turnOf file0

165. Why use Git?

Answer: Git provides version control: a history of changes, collaboration support, rollback capability and a clear record of how a codebase evolved.

166. What is a commit?

Answer: A commit is a recorded snapshot of changes in the repository history with metadata and a message describing the change.

167. Why use meaningful commits?

Answer: Meaningful commits make the history understandable and make it easier to review, debug, revert or reason about changes.

168. Why avoid one giant end-of-day commit?

Answer: A giant commit hides the progression of the work and makes review and debugging harder. Smaller coherent commits communicate engineering intent.

169. What does `feat` mean in a commit message?

Answer: It indicates a new feature or capability.

170. What does `test` mean?

Answer: It indicates changes primarily related to tests.

171. What does `refactor` mean?

Answer: It indicates restructuring code without intending to change its externally observable behavior.

172. What does `chore` mean?

Answer: It indicates maintenance or setup work that is not primarily a user-facing feature, such as initializing project structure.

173. What is the difference between local and remote repositories?

Answer: A local repository is on your development machine. A remote repository, such as one hosted on GitHub, is used to share and store the project history externally.

174. What does git push do?

Answer: It sends local commits to a configured remote repository.

175. Why does commit history matter in an interview?

Answer: It can show that the project was developed incrementally and that you understand version-control discipline rather than only presenting a final code dump.

176. What if `.venv` was accidentally committed?

Answer: Remove it from Git tracking, add it to .gitignore, and if necessary clean the repository history using an appropriate history-rewriting procedure. The exact remediation depends on whether the commit was already shared and whether sensitive material is involved.

16. Project Structure and Engineering Trade-offs

177. Why have a src directory?

Answer: It provides a clear boundary for application source code and keeps source modules separate from tests, documentation and project metadata.

178. What belongs in api/?

Answer: HTTP-facing route handlers, request/response schemas and transport-specific behavior.

179. What belongs in services/?

Answer: Application/domain service logic such as DocumentService and business operations.

180. What belongs in repositories/?

Answer: Persistence abstractions and implementations such as DocumentRepository.

181. What belongs in models/?

Answer: Domain or application data representations, depending on the project's chosen model boundaries. The Day 2 plan does not prescribe a complete model taxonomy.

182. What belongs in core/?

Answer: Cross-cutting foundational configuration or utilities, when needed. Day 2 intentionally keeps this minimal.

183. Why have tests/?

Answer: To keep automated tests separate from production application code and make the testing boundary explicit.

184. Why have docs/?

Answer: To provide a place for project documentation without mixing documentation artifacts into application source modules.

185. Why not put everything in main.py?

Answer: A single file becomes difficult to navigate and encourages route handlers to accumulate business and persistence logic. Separation supports maintainability and testing.

186. Why does the plan say not to over-engineer?

Answer: The goal is to establish a clean foundation, not to introduce every production technology immediately. Extra abstraction without a real need can add complexity without improving the current problem.

187. When is a simple architecture better?

Answer: When the system is small and the boundaries can be expressed clearly with a few focused components. Complexity should be introduced when it solves a real problem.

188. Would the structure change in a large production system?

Answer: Likely yes. More domains, route modules, configuration, infrastructure, database adapters and testing layers may be introduced. The principle of clear responsibilities should remain.

189. What is the danger of premature abstraction?

Answer: You may design around hypothetical future requirements, creating unnecessary interfaces and indirection that make the present system harder to understand.

190. What is the danger of under-structuring?

Answer: Everything becomes coupled. Business logic, HTTP details and storage concerns become difficult to change independently.

17. End-to-End Day 2 Scenarios

These questions are designed to test whether you understand the system as an engineer rather than as a collection of isolated definitions.

191. Walk me through POST /documents from request to response.

Answer: The client sends JSON to POST /documents. FastAPI matches the route and validates the request against DocumentCreate. The API layer translates the validated input into a business operation and calls DocumentService. The service applies business rules and calls DocumentRepository. The repository writes to the in-memory store. The resulting document is converted into the DocumentResponse contract and returned as an HTTP response.

192. What happens if name is missing?

Answer: The request fails API schema validation before normal business processing. The client receives a validation error rather than a successful document creation.

193. What happens if version has an invalid value?

Answer: If it violates the declared API schema, validation rejects the request. If it is structurally valid but violates a domain-specific business rule, the service layer should enforce that rule.

194. What happens if a requested document does not exist?

Answer: The service/repository path identifies that the document is absent, a DocumentNotFoundError is raised, and the API layer translates that into HTTP 404.

195. What if the repository raises an unexpected exception?

Answer: The API should not fabricate a successful response. The error should be handled according to the application's error policy and generally surface as an appropriate server-side failure.

196. Why does the API call the service rather than the repository?

Answer: The service is the boundary for business behavior. Calling it keeps business rules centralized and prevents the API from becoming coupled to persistence details.

197. Why does the service call the repository?

Answer: The service needs persistence operations but should not own the details of how data is stored. The repository provides that boundary.

198. How would you change from in-memory storage to PostgreSQL?

Answer: Introduce or replace the repository implementation with one that persists through PostgreSQL. Keep the service's business responsibilities and API contracts separate from database-specific operations.

199. What if you later expose document creation through a CLI?

Answer: The CLI can call the same service layer rather than duplicating business rules. The service and repository can be reused because they are not coupled to HTTP.

200. What if you later add a background worker?

Answer: The worker can also invoke the service layer for business operations, again avoiding duplication of domain behavior.

201. What if the API response needs a new field?

Answer: Update the response contract and the mapping needed to produce it. The database does not necessarily need to change if the information already exists or can be derived.

202. What if the database needs a new internal field?

Answer: Update persistence concerns and repository/model mapping as necessary without automatically exposing the field through the API.

203. What if a business rule is accidentally placed in the route?

Answer: Refactor it into the service/domain layer so the rule is reusable and independently testable. The route should translate HTTP input into the business operation, not own the business operation itself.

18. What You Deliberately Did Not Build

The Day 2 plan explicitly postpones PostgreSQL, Redis, Docker, Kubernetes, GCP, LangChain, LangGraph, vector databases, embeddings, RAG, agents, React, authentication and PDF processing. It explains that adding them at this stage would create breadth without establishing the engineering foundation needed for the later AI/ML system.

■filecite■turn0file0■

204. Why didn't you add PostgreSQL on Day 2?

Answer: Because Day 2 was focused on establishing the application and persistence boundary using in-memory storage. PostgreSQL is an infrastructure/persistence expansion that can be introduced after the basic architecture and tests are stable.

205. Why didn't you add Docker?

Answer: Containerization was intentionally postponed so the day could focus on understanding the application architecture and API behavior rather than deployment tooling.

206. Why didn't you add RAG or vector databases?

Answer: They are later AI-system concerns. Day 2 first establishes a reliable backend foundation into which those capabilities can eventually be integrated.

207. Why didn't you add agents?

Answer: Agent orchestration was intentionally postponed. The project first needs clean API, service, persistence and testing boundaries.

208. Why didn't you add authentication?

Answer: Authentication is a separate cross-cutting concern and was explicitly excluded from Day 2 to keep the scope focused.

209. Does postponing these technologies mean they are unimportant?

Answer: No. It means they are intentionally sequenced later. The Day 2 strategy is to build depth in the engineering foundation before expanding the technology stack.

210. What would you add next if the foundation were stable?

Answer: The exact next step should follow the broader career blueprint. Conceptually, you could replace the in-memory repository with durable persistence, add stronger configuration/testing infrastructure, and later introduce the AI/ML components once the backend foundation is ready.

19. High-Priority Mock Interview

These are the questions I would prioritize if you had limited time before an interview. Your answers should be conversational, not memorized word-for-word.

211. Walk me through what you built on Day 2.

Answer: I took the Python business logic from Day 1 and exposed it through a clean FastAPI backend. I added the project structure, virtual environment, FastAPI endpoints, Pydantic request/response schemas, service/repository integration, error handling, API tests, and the Two Pointer DSA problems.

212. Explain your architecture.

Answer: The architecture is HTTP Client → FastAPI API layer → DocumentService → DocumentRepository → in-memory storage. FastAPI handles transport concerns, the service owns business behavior, and the repository owns persistence.

213. Why not put everything in the route?

Answer: That would couple business logic to HTTP. It would make reuse and testing harder and cause routes to become large and difficult to maintain.

214. What is the role of Pydantic?

Answer: Pydantic gives the API an explicit data contract and validates incoming data. It makes request and response structures clear rather than relying on raw dictionaries.

215. Why separate DocumentCreate and DocumentResponse?

Answer: The client input and server output have different responsibilities. The server can generate fields such as id and status, so they should not necessarily be part of the create request.

216. How does a Python exception become HTTP 404?

Answer: The domain/application code raises DocumentNotFoundError. The API layer translates that application-level condition into the HTTP-level representation, 404 Not Found.

217. How do you test the API?

Answer: I test the success paths for GET /, GET /health and POST /documents, plus validation failure and missing-document behavior. The tests assert both status codes and response data.

218. Explain Two Sum II.

Answer: Because the array is sorted, I use left and right pointers. If the sum is too small I move left to increase the sum; if it is too large I move right to decrease the sum. Each pointer moves only inward, giving $O(n)$ time and $O(1)$ extra space.

219. Explain Best Time to Buy and Sell Stock.

Answer: I scan once, tracking the minimum price seen so far and the best profit so far. At each price, the best transaction ending there uses the minimum earlier price. This gives $O(n)$ time and $O(1)$ extra space.

220. Why did you avoid PostgreSQL, Docker and RAG?

Answer: They were intentionally postponed. The Day 2 objective was to establish clean application boundaries and tests first, so later infrastructure and AI capabilities can be added without creating a tangled system.

20. Rapid-Fire Revision

221. What is FastAPI?

Answer: A Python web framework for building APIs.

222. What is Uvicorn?

Answer: An ASGI server used to run FastAPI applications.

223. What is ASGI?

Answer: The interface between Python async-capable web applications and servers.

224. What is an endpoint?

Answer: A method/path API operation.

225. What is Pydantic?

Answer: A typed data validation/modeling library integrated with FastAPI.

226. What is a request model?

Answer: The schema for data entering the API.

227. What is a response model?

Answer: The schema for data leaving the API.

228. What is validation?

Answer: Checking whether input satisfies an expected contract.

229. What is serialization?

Answer: Converting an in-memory representation to a transport/storage representation.

230. What is deserialization?

Answer: Converting external data into an in-memory representation.

231. What is a service layer?

Answer: The layer that owns application/business behavior.

232. What is a repository layer?

Answer: The layer that owns persistence/storage operations.

233. Why separate API and service?

Answer: To keep transport concerns separate from business behavior.

234. Why separate service and repository?

Answer: To keep business behavior separate from persistence.

235. What is HTTP 404?

Answer: Resource not found.

236. What is HTTP 4xx?

Answer: A request/client-side error category.

237. What is HTTP 5xx?

Answer: A server-side failure category.

238. What is pytest?

Answer: A Python testing framework.

239. What is a fixture?

Answer: Reusable test setup/data.

240. What is a virtual environment?

Answer: An isolated Python package environment.

241. Why not commit .venv?

Answer: It is generated, environment-specific state.

242. What is requirements.txt?

Answer: A declaration/snapshot of project dependencies.

243. What is Two Pointers?

Answer: An algorithmic pattern using two indices/references to reduce unnecessary work.

244. Why does Two Sum II use Two Pointers?

Answer: Because sorted order tells you which pointer movement can increase or decrease the sum.

245. Why is Valid Palindrome Two Pointer-friendly?

Answer: The palindrome property compares mirrored characters from both ends.

246. What does stock optimization track?

Answer: Minimum price seen and best profit so far.

247. What is an invariant?

Answer: A property that remains true during an algorithm.

248. Why automate API tests?

Answer: To make API behavior repeatable and regression-resistant.

249. Why meaningful Git commits?

Answer: To communicate coherent changes and maintain useful history.

250. Why keep in-memory storage for now?

Answer: It keeps the foundation simple while preserving a repository boundary for later persistence.

21. Day 2 Final Revision Checklist

Use this as the final self-assessment. Do not mark an item complete because you recognize the words. Mark it complete only if you can explain it aloud and, where relevant, implement it.

Backend setup

- Explain why a virtual environment is used.
- Explain requirements.txt and pip freeze.
- Explain why .venv belongs in .gitignore.
- Explain the project directory structure.
- Explain why the project should not be over-engineered.

FastAPI

- Define FastAPI.
- Define Uvicorn and ASGI.
- Explain uvicorn src.main:app --reload.
- Explain GET /.
- Explain GET /health.
- Explain POST /documents.
- Explain /docs and Swagger UI.
- Explain request body, path parameter and query parameter.
- Explain HTTP status codes relevant to Day 2.

Pydantic

- Define Pydantic.
- Explain request validation.
- Explain DocumentCreate.
- Explain DocumentResponse.
- Explain why request and response models are separate.
- Explain serialization/deserialization.
- Explain API schema vs business object vs database model.

Architecture

- Draw API → Service → Repository → Storage from memory.
- Explain the responsibility of each layer.

- Explain why business rules do not belong in route handlers.
- Explain why repositories should not know about HTTP.
- Explain how the architecture supports changing storage later.
- Explain dependency injection conceptually.
- Explain how each layer can be tested.

Errors and tests

- Explain Python exception vs HTTP error.
- Explain `DocumentNotFoundError` → 404.
- Explain invalid input handling.
- Explain unexpected failure handling.
- Explain pytest.
- Explain fixtures and assertions.
- Explain API tests for success, validation and not-found behavior.
- Explain unit vs integration/API tests.

DSA

- Explain the Two Pointer pattern.
- Recognize when it applies.
- Solve Valid Palindrome.
- Solve Two Sum II.
- Solve Best Time to Buy and Sell Stock.
- Explain brute force and optimized approaches.
- Give time and space complexity.
- Explain why each optimization is correct.
- State one mistake/observation for each problem.

Git and engineering

- Explain meaningful commit messages.
- Explain feat/refactor/test/chore.
- Explain local vs remote repository.
- Explain why commit history matters.
- Explain how to verify README, source tree and tests on GitHub.

Scope discipline

- Explain why PostgreSQL was postponed.
- Explain why Docker/Kubernetes were postponed.
- Explain why RAG/vector databases/embeddings were postponed.
- Explain why agents were postponed.
- Explain why authentication and frontend were postponed.
- Explain why a clean foundation comes before breadth.

Final interview statement to memorize conceptually

“Day 2 was about turning yesterday’s Python business logic into a real backend service. I used FastAPI as the HTTP boundary, Pydantic for explicit request/response contracts and validation, a service layer for business behavior, and a repository layer for persistence. I added deliberate error translation and automated API tests, while keeping storage in memory to avoid premature infrastructure. On the DSA side, I practiced pattern recognition with Two Pointers through Valid Palindrome, Two Sum II and Best Time to Buy and Sell Stock. The main lesson was separation of concerns and being able to explain why each layer and optimization exists.”

Source basis: Google Career Blueprint — Day 2 — Detailed Execution Plan & Checklist, 11 September 2026. The uploaded document contains the Day 2 objectives, time blocks, project structure, FastAPI requirements, Pydantic concepts, architecture, error handling, API tests, DSA problems, Git discipline, definition of done, strategy adjustment and engineering takeaways. ■filecite■turn0file0■